

Convolutional neural network

Algorithm:

- 1) Padding (add border for image)
- 2) Convolve image with kernel (3×3)
- 3) Activation function (ReLU)
- 4) Max pooling (2×2)
- 5) Repeat this operation
- 6) Transfer output information to fully-connected neural network (49 input parameters)
- 7) Multiply matrices ($W_1 \cdot X + b_1$, W -weights, X -input data, b -biases)
- 8) Use ReLU
- 9) Multiply matrices ($W_2 X + b_2$)

10) Use ReLU

11) Multiply matrices ($W_3X + b_3$)

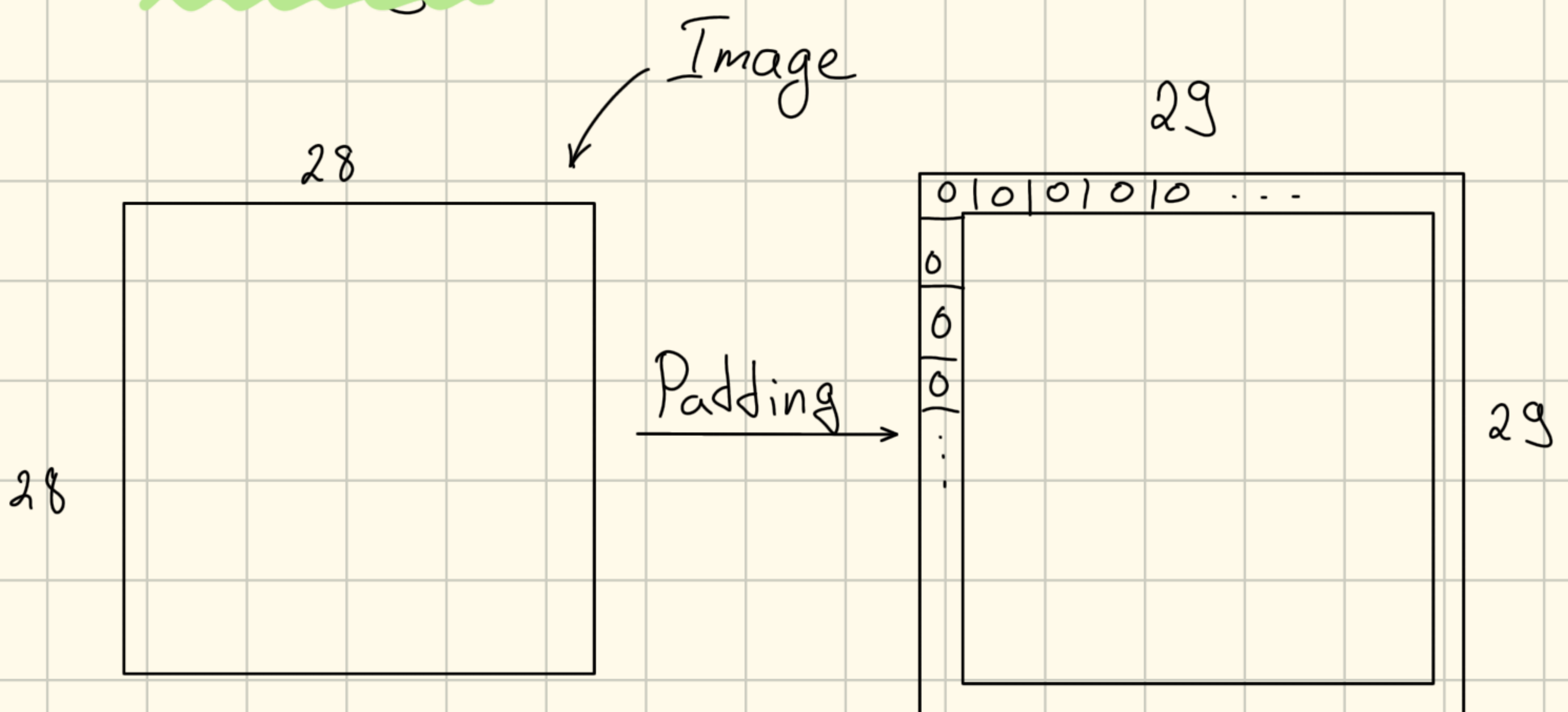
12) Use Softmax

13) Calculate loss function (categorical cross-entropy)

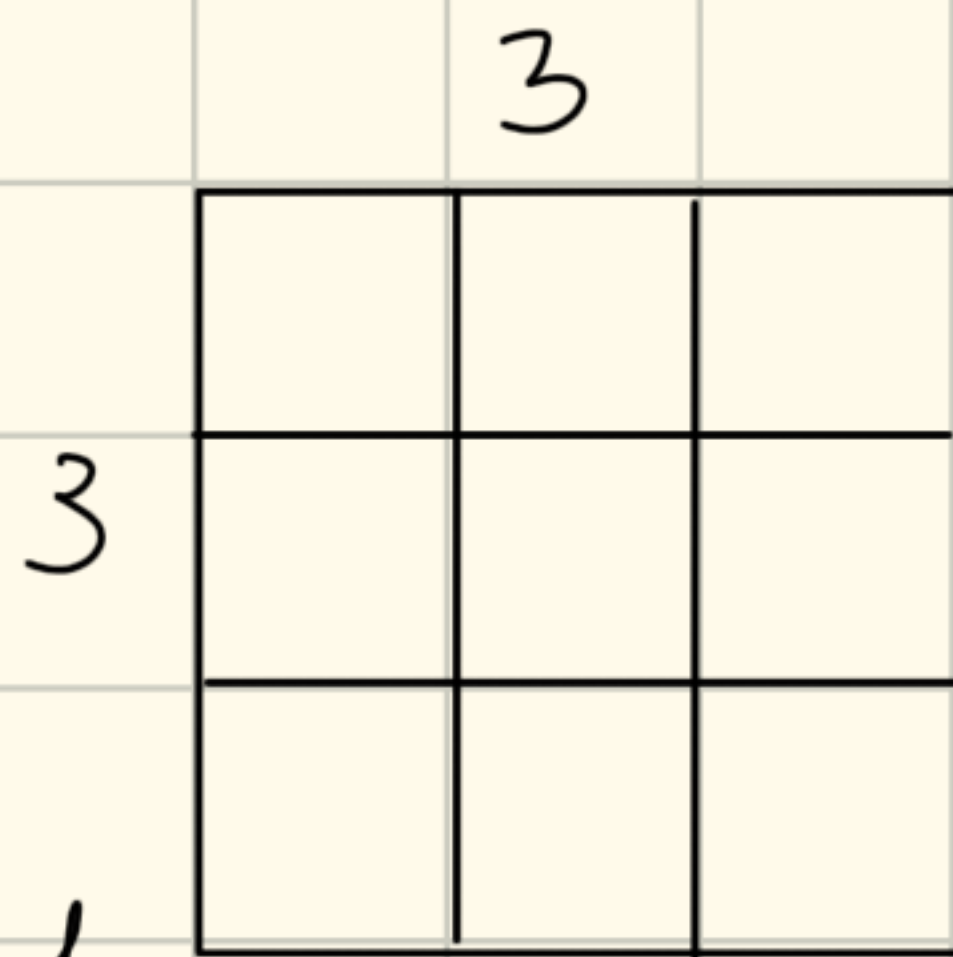
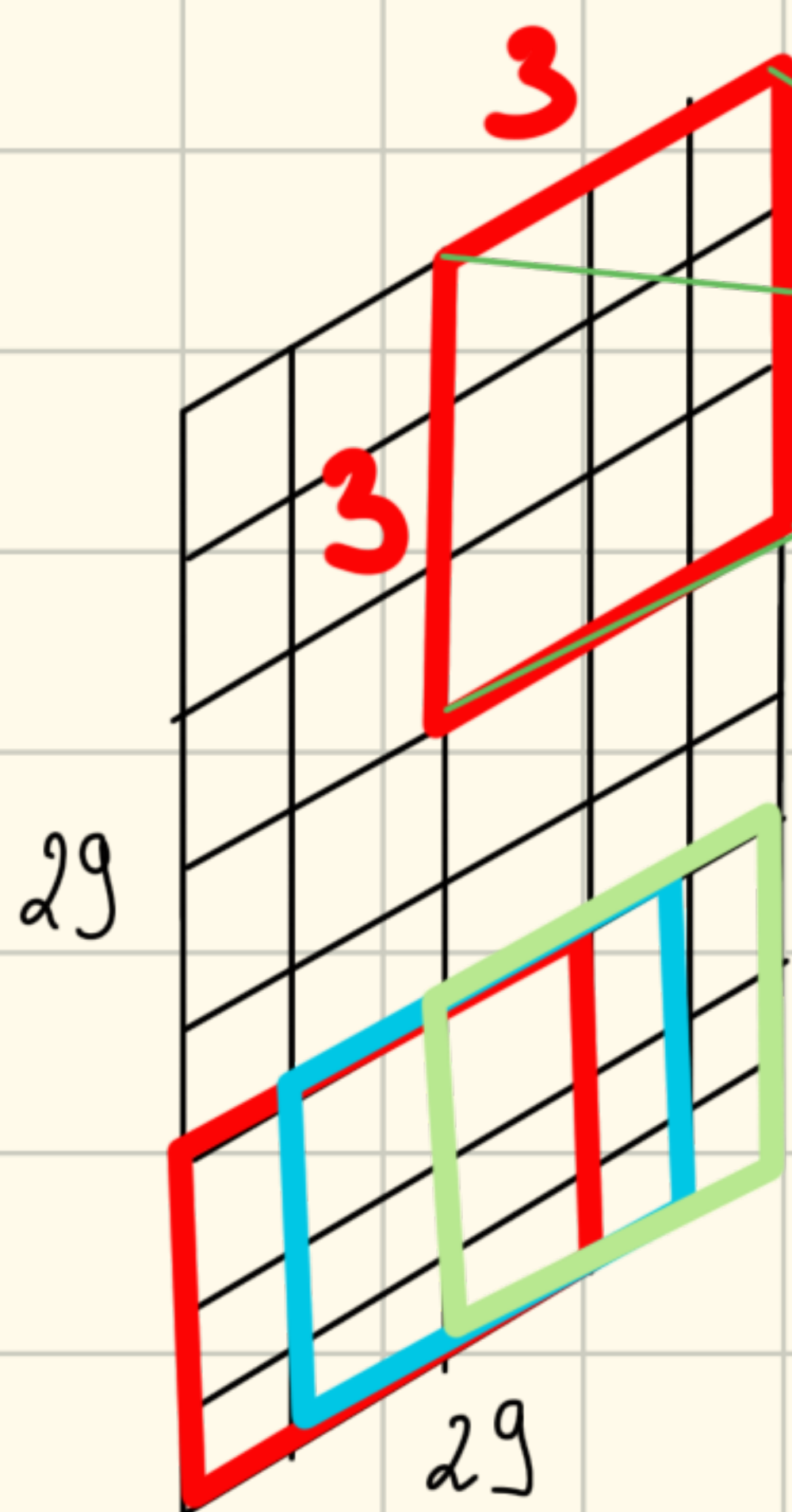
14) Backward propagation to calculate gradient and use it to update weights and biases ($W_1, W_2, W_3, b_1, b_2, b_3$)

15) Train network to optimize loss function

Padding:



Convolve:



← Kernel (K)



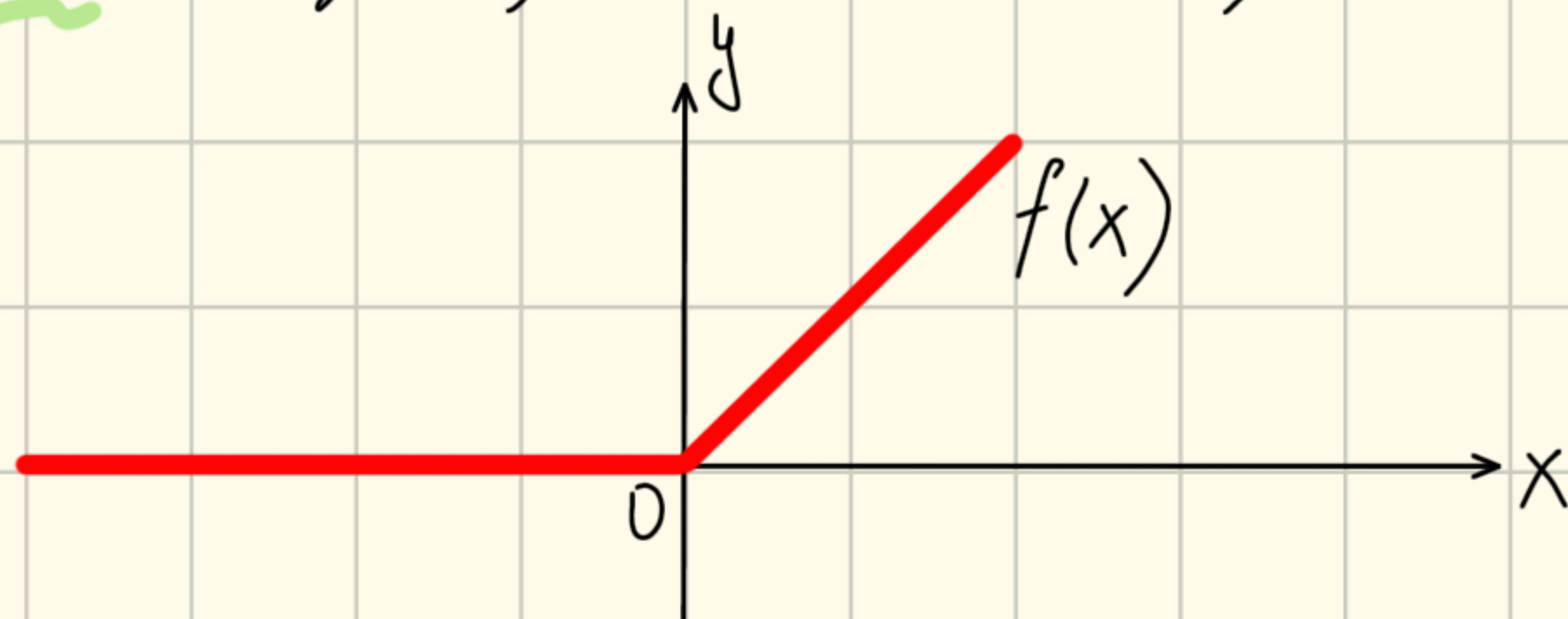
← new value

Convolve every block of pixels (3x3) with kernel:

$$X = \begin{pmatrix} p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 \\ p_7 & p_8 & p_9 \end{pmatrix}, \quad K = \begin{pmatrix} k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 \\ k_7 & k_8 & k_9 \end{pmatrix}$$

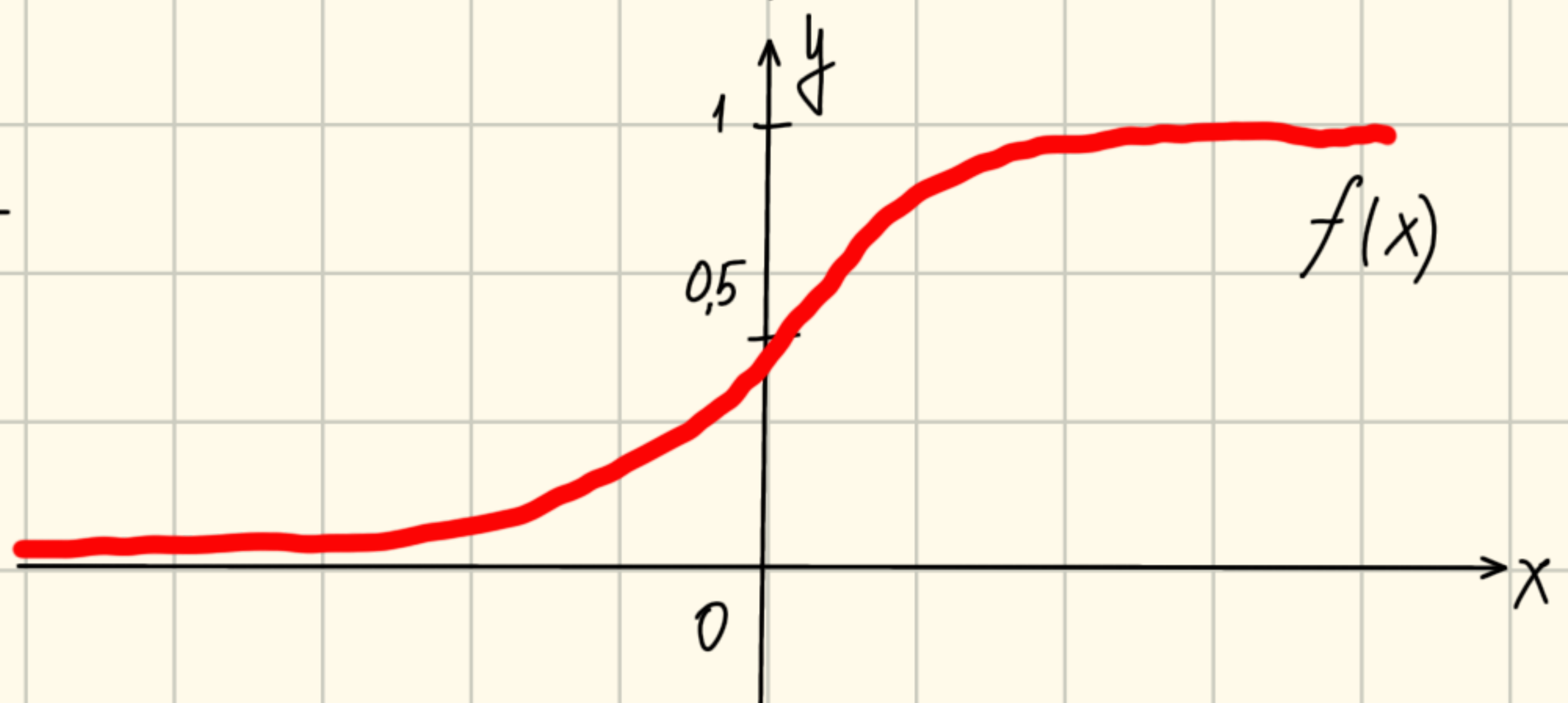
$$\text{new value} = p_1 \cdot k_1 + p_2 \cdot k_2 + \dots + p_9 \cdot k_9 = \sum_{i=1}^9 p_i \cdot k_i$$

ReLU: $f(x) = \max(0, x)$

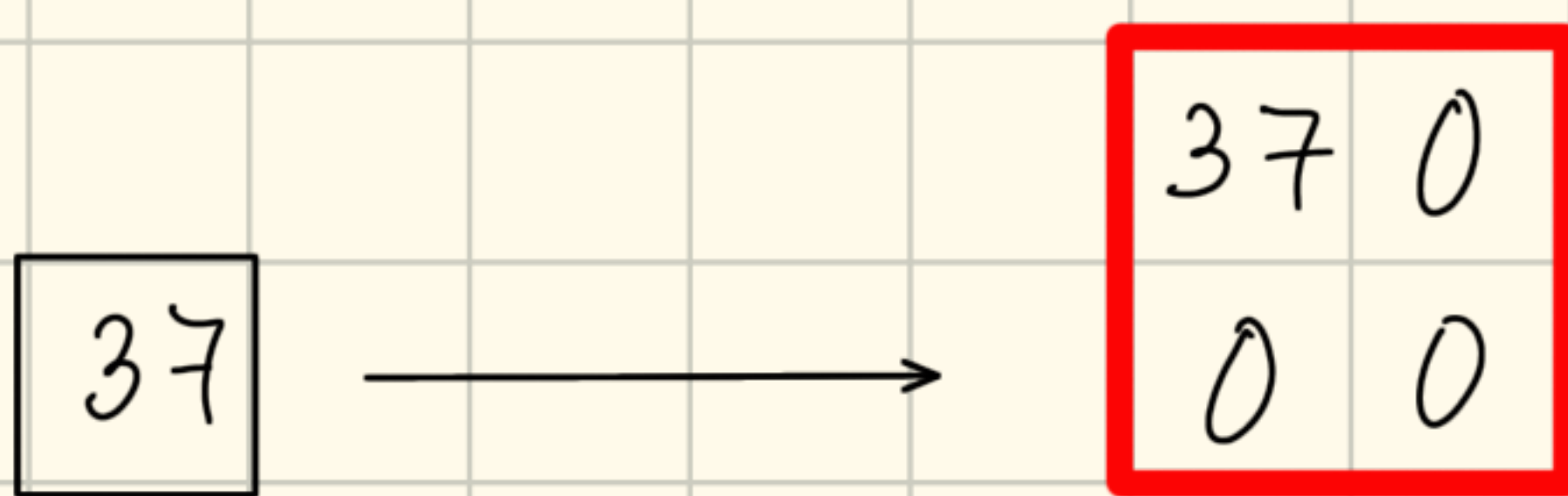
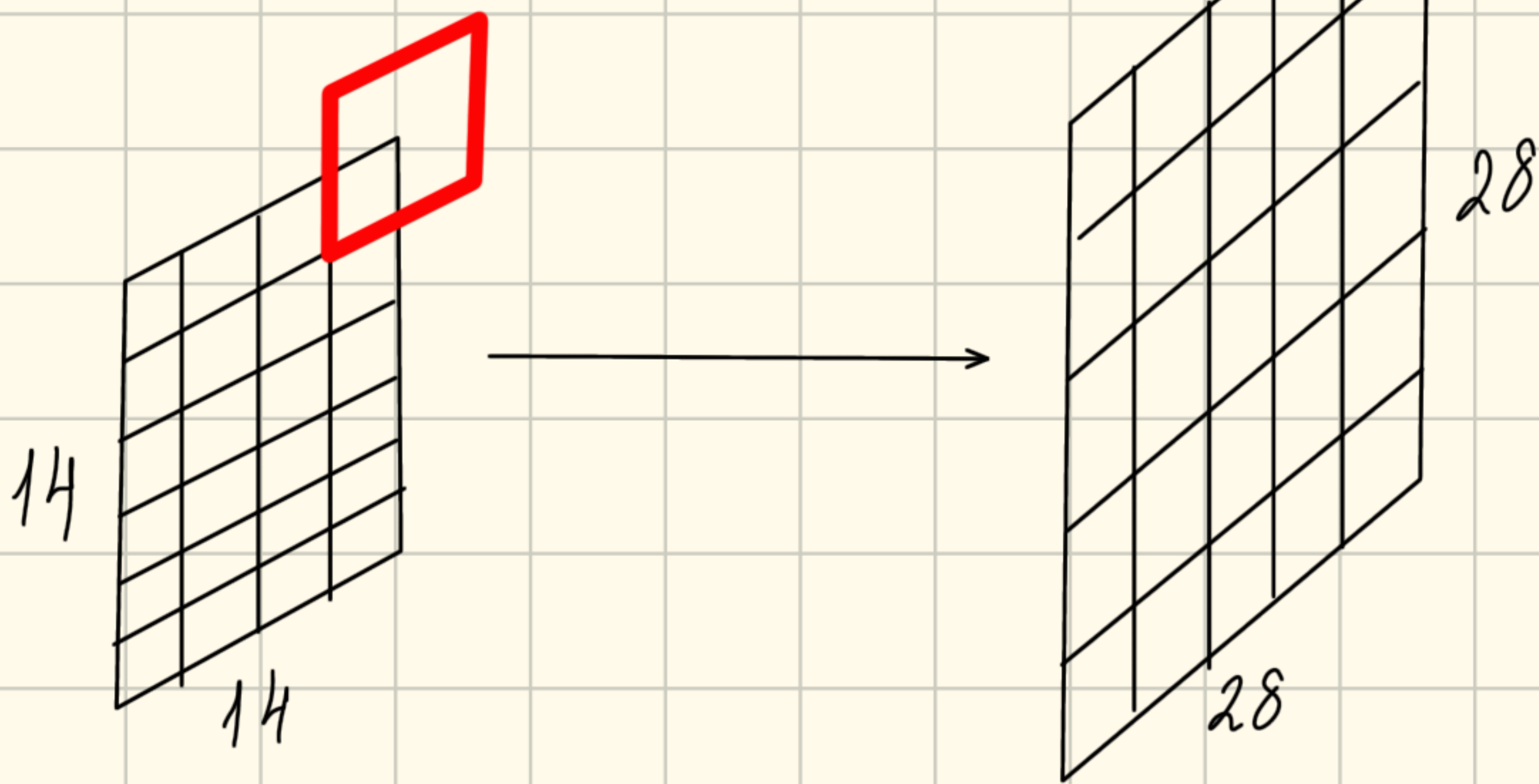


Softmax: $X = (x_1, x_2, \dots, x_n)$

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



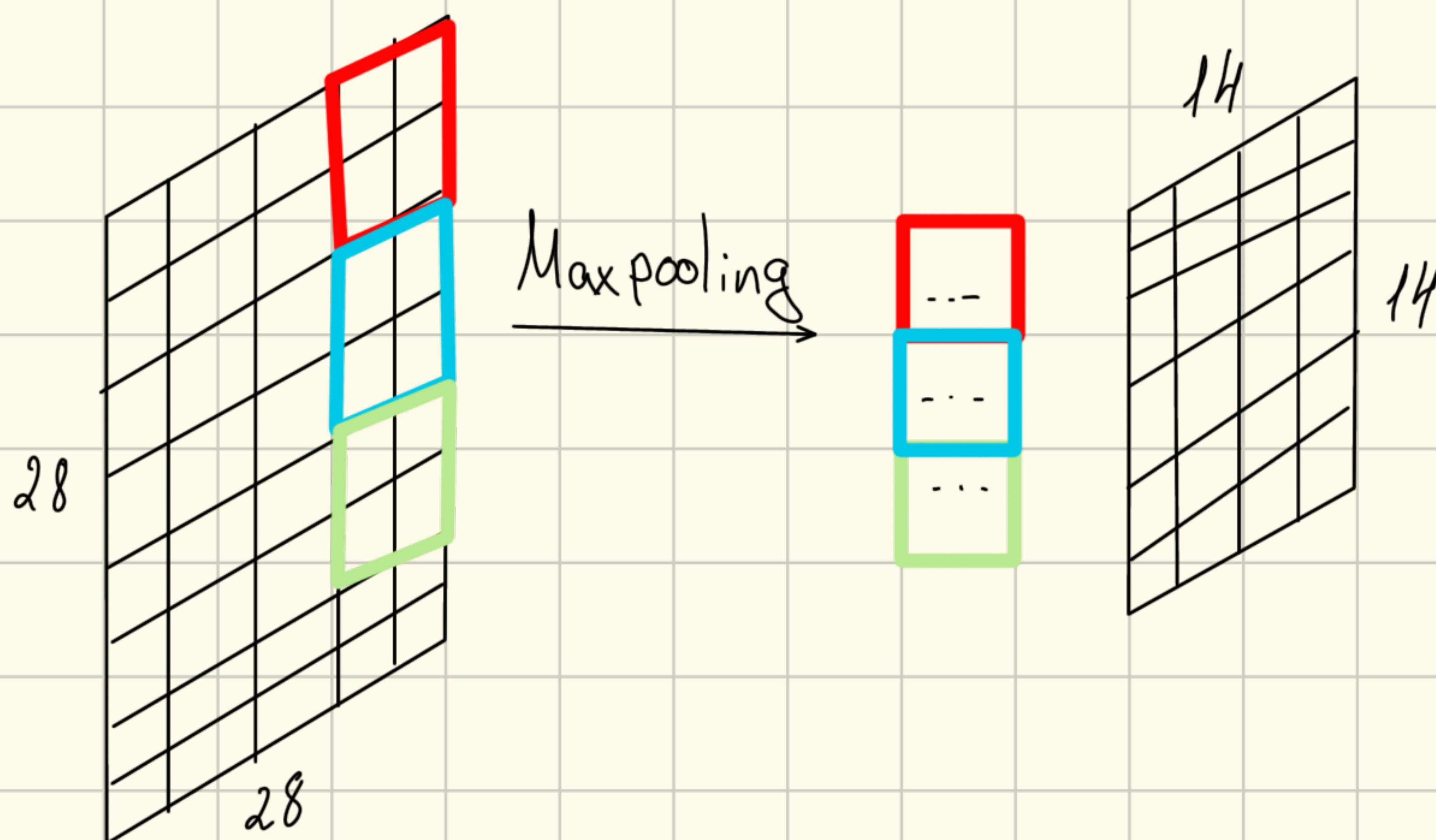
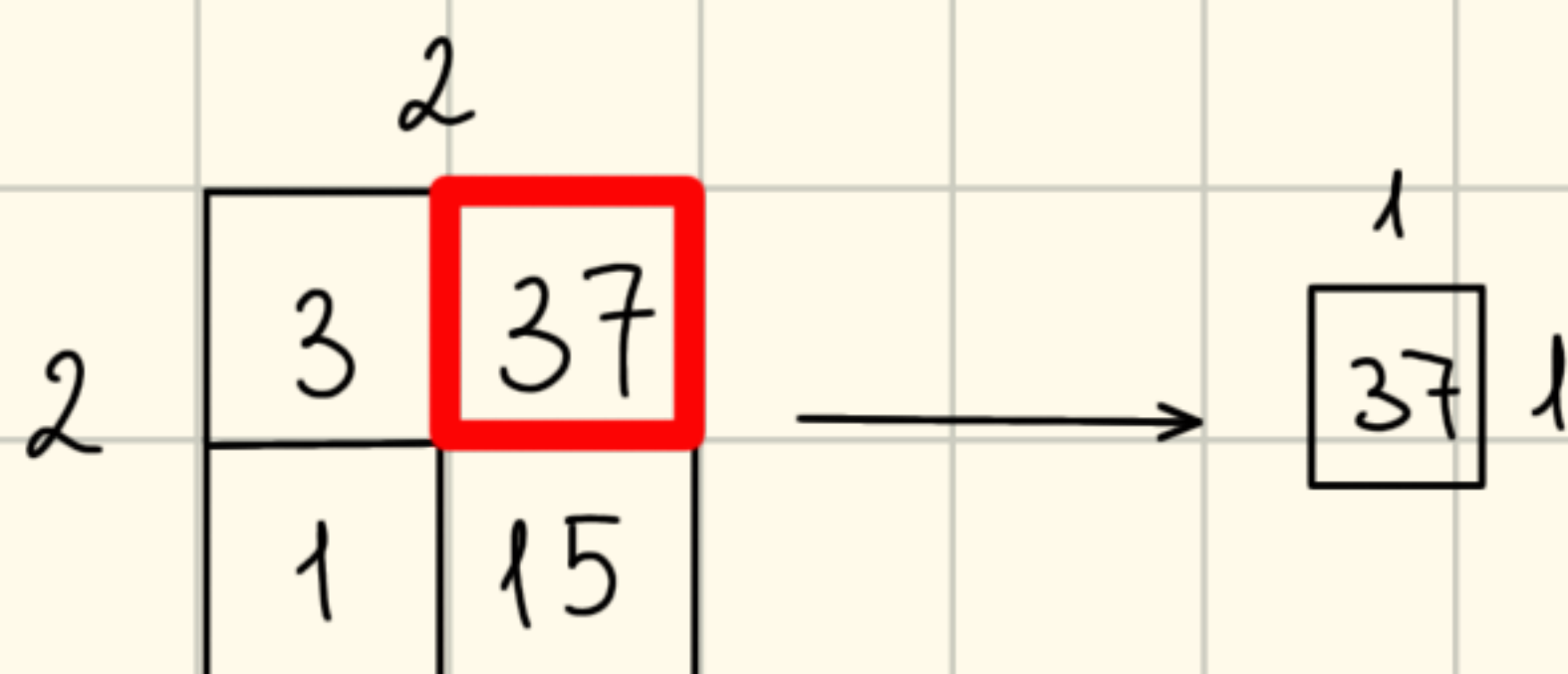
Zero filling:



Expand the image by adding zeros

Max pooling:

Take the results after convolution and activation function and select maximum value.



Loss function:

$y = (y_0, y_1, \dots, y_9)$ — истинные значения

$\tilde{y} = (\tilde{y}_0, \tilde{y}_1, \dots, \tilde{y}_9)$ — вычисленные значения

$$L(y_i, \tilde{y}_i) = - \sum_{i=0}^9 y_i \cdot \log(\tilde{y}_i + \epsilon), \quad \epsilon -$$

научное число ($\approx 10^{-8}$), чтобы избежать $\log(0)$

Forward propagation:

t_i - intermediate result, W_i - weights, b_i - biases,
 X' - input data (1×49)

$$t_1 = X' \cdot W_1 + b_1, \quad \left. \begin{array}{l} \dim W_1 = 49 \times 128 \\ \dim b_1 = 1 \times 128 \end{array} \right\} \Rightarrow$$

$$\Rightarrow \dim t_1 = 1 \times 128$$

$$h_1 = \text{ReLU}(t_1), \quad \dim h_1 = 1 \times 128$$

$$t_2 = h_1 W_2 + b_2, \quad \left. \begin{array}{l} \dim W_2 = 128 \times 10 \\ \dim b_2 = 1 \times 10 \end{array} \right\} \Rightarrow$$

$$\Rightarrow \dim t_2 = 1 \times 10$$

$$z = \text{Softmax}(t_2)$$

$$L = \sum (\text{cross-entropy}(z, y)), \quad \underset{\text{answer}}{y} - \text{right}$$

Backward propagation;

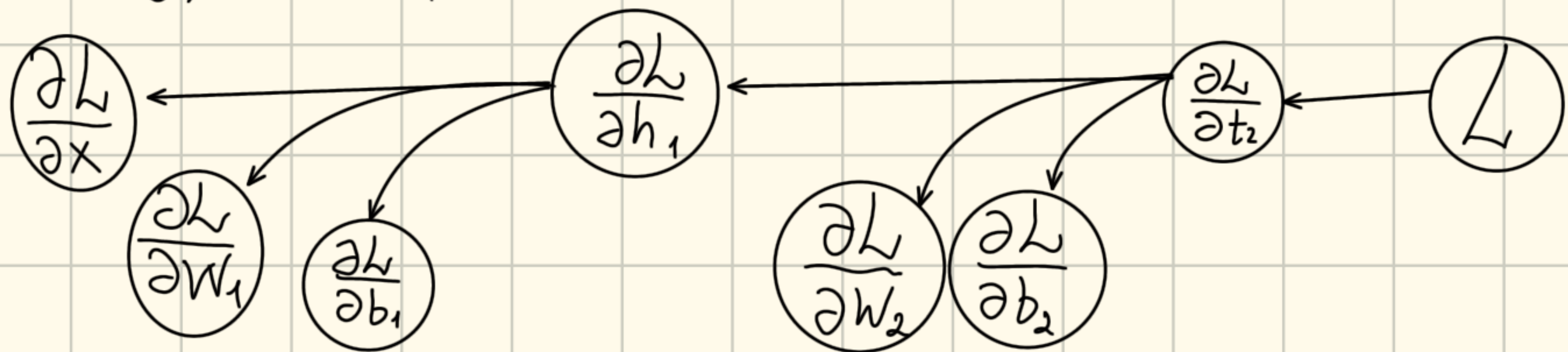
$$\frac{\partial L}{\partial t_2} = z - y, \quad y = (0, 0, \dots, 1, 0, \dots, 0)$$

$$\frac{\partial L}{\partial w_2} = h_1^T \cdot \frac{\partial L}{\partial t_2}, \quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial t_2}, \quad \frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial t_2} \cdot w_2^T,$$

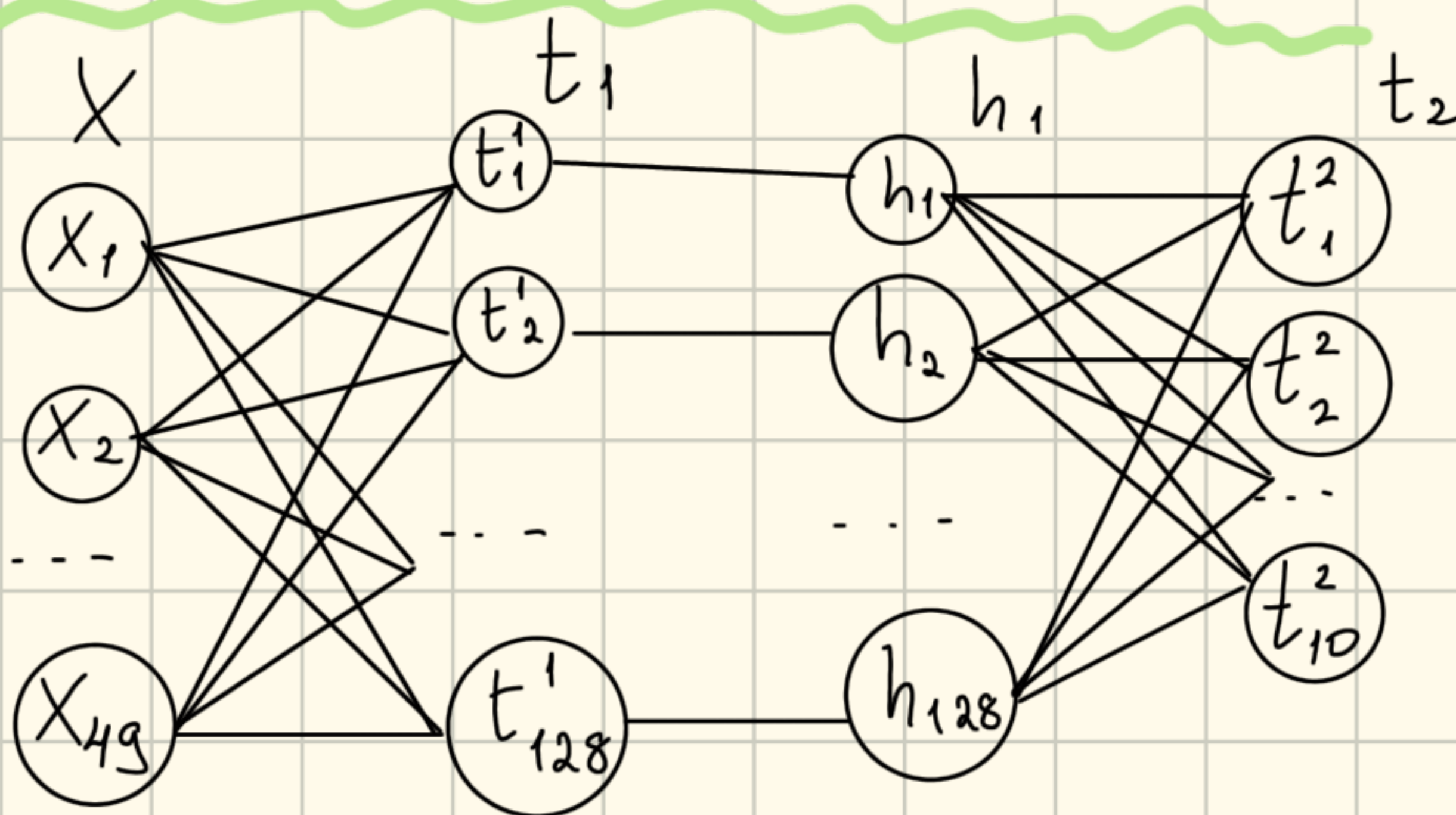
$$\frac{\partial L}{\partial t_1} = \frac{\partial L}{\partial h_1} \odot F'(t), \quad \odot - \text{element-wise multiplication, } F(x) = \text{ReLU}(x),$$

$$\frac{\partial L}{\partial w_1} = x^T \cdot \frac{\partial L}{\partial t_1}, \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial t_1}$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial t_1} \cdot w_1^T$$



Fully-connected network:



Convolution: @ - convolve

X - input data (28×28), K_1, K_2 - kernels (3×3)

Forward propagation:

$$T_1 = X @ K_1, \dim T_1 = 28 \times 28$$

$$H_1' = \text{ReLU}(T_1)$$

$$H_1 = \text{MP}(H_1'), \text{MP} - \text{Max pooling}, \dim H_1 = 14 \times 14$$

$$T_2 = H_1 @ K_2, \dim T_2 = 14 \times 14$$

$$H_2' = \text{ReLU}(T_2)$$

$$H_2 = \text{MP}(H_2')$$

Backward propagation: $\text{IF} = \text{ReLU}$

$$\frac{\partial L}{\partial H_2} = \frac{\partial L}{\partial X}, \frac{\partial L}{\partial H_2'} = ZF\left(\frac{\partial L}{\partial H_2}\right), ZF - \text{zerofilling}$$

$$\frac{\partial L}{\partial T_2} = \frac{\partial L}{\partial H_2'} \odot F'(T_2), \frac{\partial L}{\partial H_1} = \frac{\partial L}{\partial T_2} @ K_2^{TT}, TT - \text{rotate } 180 \text{ degrees}$$

$$\frac{\partial L}{\partial K_2} = H_1 @ \frac{\partial L}{\partial T_2}, \frac{\partial L}{\partial H_1'} = ZF\left(\frac{\partial L}{\partial H_1}\right), \frac{\partial L}{\partial T_1} = \frac{\partial L}{\partial H_1'} \odot F'(T_1),$$

$$\frac{\partial L}{\partial K_1} = X @ \frac{\partial L}{\partial T_1}$$

